Client-Server Paradigm

Question & Answers with Diagrams

Computer Networks

Transport & Application Layer

Syllabus Overview

Transport Layer Foundations

Section	What It Covers
6.1.1	Introduction to process-to-process delivery and the client-server model at the transport layer

Connectionless Client-Server (UDP)

Section	What It Covers
6.4 up to 6.4.1	Introduction to UDP - how client-server communication works without establishing a connection

Connection-Oriented Client-Server (TCP)

Section	What It Covers
6.5.3	The TCP Protocol - how client-server communication works with a reliable connection
6.5.4	The TCP Segment Header - the format of data units exchanged between client and server

Application-Layer Client-Server Examples

Section	What It Covers
7.1	DNS - The client-server architecture of the Domain Name System
7.2.4	Message Transfer (SMTP) - How email clients and servers communicate
7.3 - 7.3.1 (till pg. 652, before MIME Type)	The World Wide Web & Architectural Overview - The web's client-server architecture
7.3.4 (before Message Headers)	HTTP - The protocol between web clients (browsers) and web servers

Paper 1: CN2 23-24 (Question Paper 3377)

Q3. (b) What is the purpose of File Transfer Protocol (FTP)? Briefly explain the different transmission modes of FTP. (6)

Answer:

File Transfer Protocol (FTP) is a standard network protocol designed for transferring files between a client and a server over TCP/IP-based networks such as the internet. It operates using two separate channels: a **control connection** (typically on port 21) for sending commands and receiving replies, and a **data connection** (on port 20) for the actual file transfer. FTP enables users to upload, download, rename, delete files, and manage directories on remote servers.

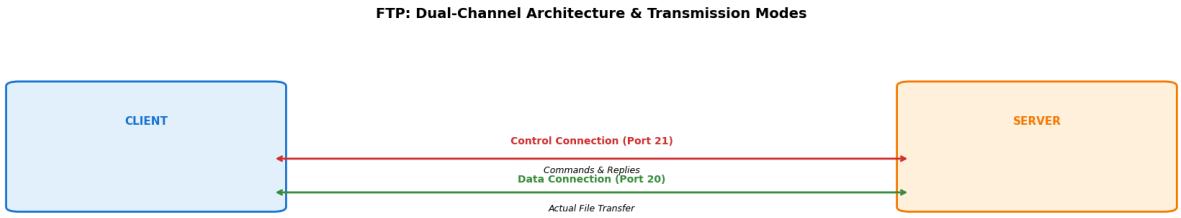

Figure 1: *FTP Dual-Channel Architecture and Transmission Modes*

Transmission Modes of FTP

FTP defines three distinct transmission modes that determine how data is packaged and sent over the data connection:

1. Stream Mode (MODE S)

This is the **default** and simplest transmission mode. Data is transmitted as a continuous stream of bytes, with FTP performing minimal processing. All segmentation and reassembly are handled by the underlying TCP. For file-structured data, the end of the file is indicated by the sender closing the data connection. For record-structured files, special two-byte control codes are embedded to mark end-of-record (EOR) and end-of-file (EOF).

2. Block Mode (MODE B)

In this mode, the file is transmitted as a series of discrete **data blocks**, each preceded by a three-byte header. The header contains a **count field** (indicating the length of the block) and a **descriptor code** that specifies whether the block is the end of a record (EOR), end of the file (EOF), a restart marker, or suspect data. Block mode supports error recovery and allows interrupted transfers to be restarted, making it suitable for record-oriented files and reliable large data transfers.

3. Compressed Mode (MODE C)

This mode extends block mode by adding **data compression** using run-length encoding to reduce redundancy. It can compress three types of information: regular data strings, replicated bytes (a sequence of identical bytes is replaced by a 2-byte code), and filler strings (sequences of spaces or zeros). While this reduces bandwidth usage for large or repetitive files (such as printer files), it requires additional CPU

processing at both ends. In practice, this mode is rarely used today because compression is often handled by other layers or pre-compressed file formats.

Q4. (a) What do you understand by Domain Name System (DNS)? Which protocol does it use to resolve the name mapping? Discuss how domain name structure is organized. (6)

Answer:

Domain Name System (DNS) is a distributed, hierarchical naming system that translates human-readable domain names (like www.example.com) into machine-readable IP addresses (like 192.168.1.1 or 2001:0db8::1). It acts as the "phonebook of the internet," allowing users to access websites using memorable names rather than numerical addresses. DNS is decentralized across millions of servers worldwide, ensuring scalability, fault tolerance, and efficient resolution.

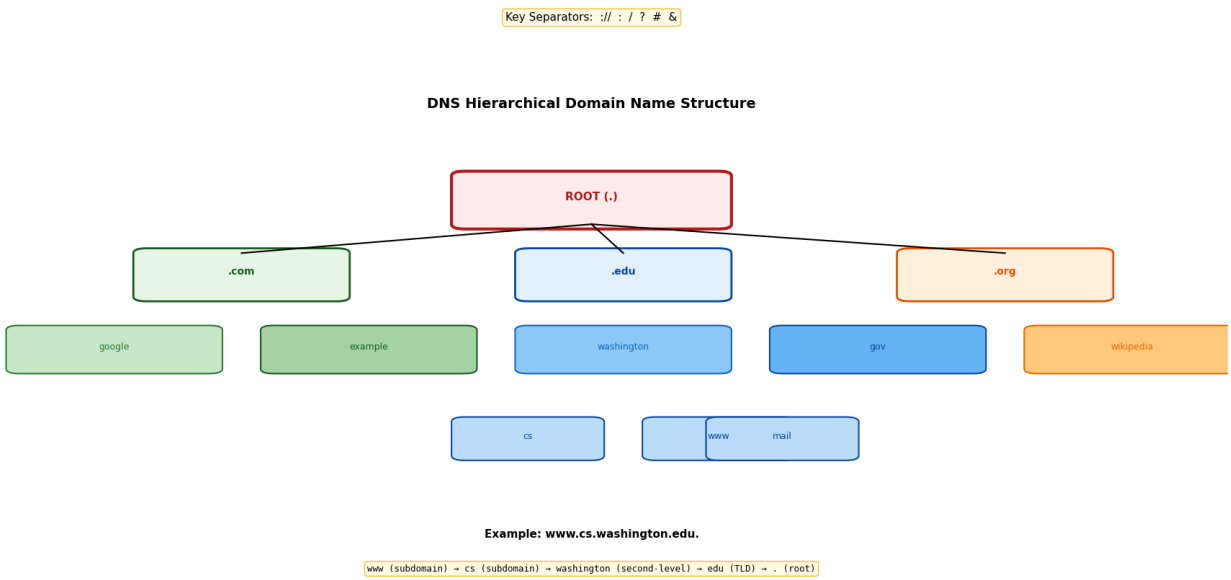

Figure 2: *DNS Hierarchical Domain Name Structure*

Protocol Used for Name Resolution

DNS primarily uses **UDP (User Datagram Protocol) on port 53** for standard name resolution queries because it is faster and has lower overhead than TCP. However, **TCP on port 53** is used for:

- **Zone transfers** (replicating DNS records between primary and secondary servers)
- **Responses exceeding 512 bytes** (modern DNS uses TCP when UDP packet size is insufficient)
- **DNSSEC-secured queries** where larger authenticated responses are required

Organization of Domain Name Structure

1. Root Domain (.)

- The highest level of the hierarchy, represented by an implied dot at the end of every fully qualified domain name (FQDN).
- Managed by root name servers that know the addresses of all Top-Level Domain (TLD) servers.

2. Top-Level Domains (TLDs)

- Located directly below the root (e.g., .com, .org, .edu, .gov, .net, .in, .uk).
- Managed by designated organizations and registries. They are broadly categorized as:
  - **Generic TLDs (gTLDs):** .com, .org, .net, .info
  - **Country-code TLDs (ccTLDs):** .in (India), .uk (United Kingdom), .jp (Japan)
  - **Sponsored TLDs (sTLDs):** .edu, .gov, .mil

3. Second-Level Domains

- Registered by organizations or individuals under a TLD (e.g., google.com, gov.in, ac.uk).
- This is typically the "organization name" purchased from a domain registrar.

4. Subdomains (Third-Level and Beyond)

- Created by the domain owner to organize services or departments (e.g., mail.google.com, www.gov.in, cs.mit.edu).
- Administrators can create multiple subdomains without additional registration.

Example Breakdown

www.example.com.

| | | |

| | | Root (.)

| | TLD (.com)

| Second-Level (example)

Subdomain (www)

This hierarchical structure allows DNS to be **distributed and delegated**: each level only needs to maintain information about the level below it, making the system scalable and manageable across the entire internet.

Q4. (b) What are the components of a URL? And what punctuation is used to separate the components? (4)

Answer:

A URL (Uniform Resource Locator) consists of the following main components, separated by specific punctuation marks:

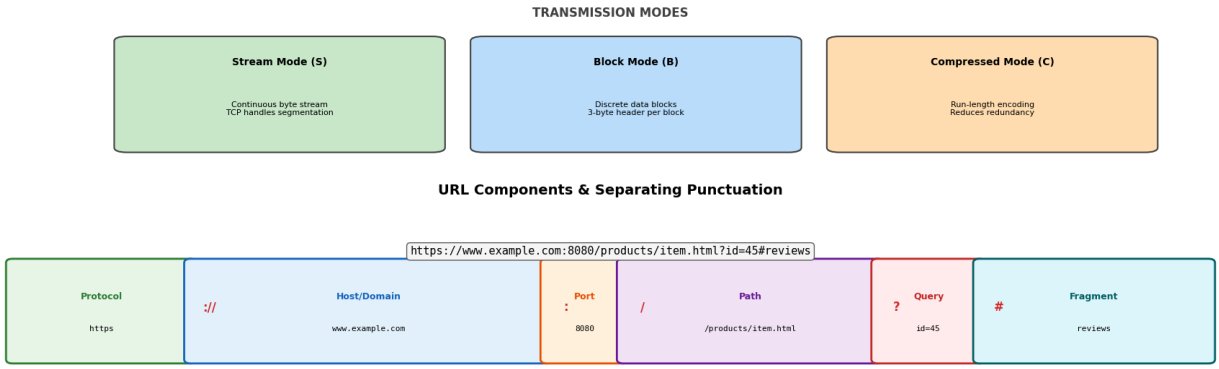

Figure 3: *URL Components and Separating Punctuation*

Components and Separating Punctuation

1. Protocol (Scheme)

- Identifies the access method (e.g., http, https, ftp, mailto)
- Followed by `://`

2. Host / Domain Name

- Identifies the server (e.g., www.example.com)
- Preceded by `://` and followed by `/` (or `:` if a port is specified)

3. Path / Resource

- Specifies the location of the file or resource on the server (e.g., /folder/index.html)
- Preceded by `/`

4. Additional Optional Components

- **Port number**: Separated from the host by a colon `:` (e.g., :8080)
- **Query string**: Separated from the path by a question mark `?` (e.g., ?id=5)
- **Fragment/Anchor**: Separated by a hash `#` (e.g., #section1)

Example Breakdown

`https://www.example.com:8080/products/item.html?id=45#reviews`

The separators are: `://` between Protocol and Host, `:` between Host and Port, `/` before the Path, `?` between Path and Query, `#` between URL and Fragment, and `&` between multiple query parameters.

Q6. (a) List the protocols that are used by modern E-mail Application system. Briefly discuss the role played by these protocols in the E-mail system. (6)

Answer:

Modern email systems rely on a suite of protocols to handle composition, transmission, retrieval, and security. Here are the key protocols and their roles:

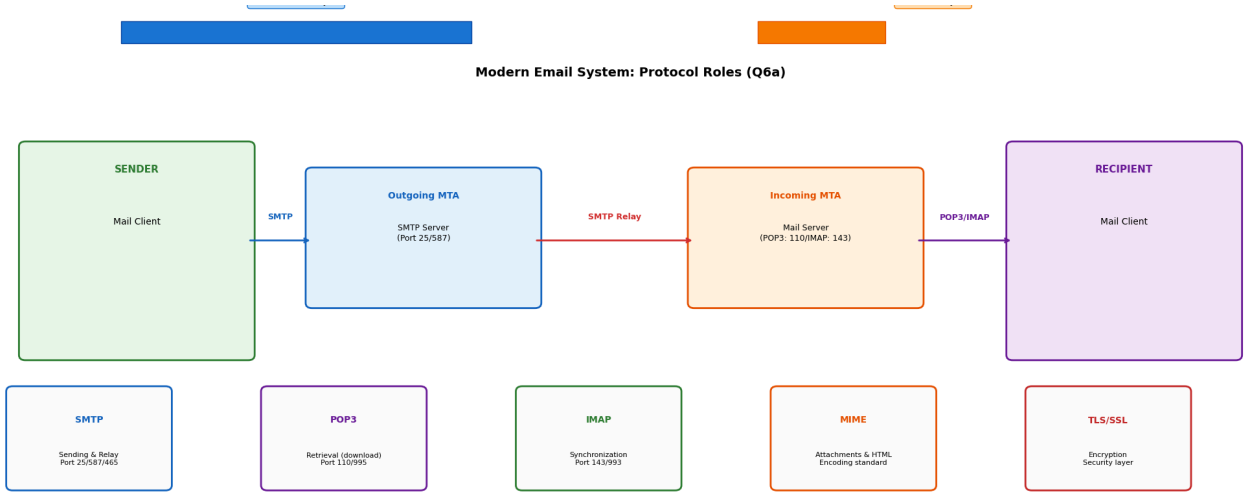

Figure 4: *Modern Email System: Protocol Roles*

1. SMTP (Simple Mail Transfer Protocol)

Role: The fundamental protocol for **sending and relaying** email messages. When a user sends an email, SMTP is responsible for transferring the message from the sender's mail client to their outgoing mail server, and then between mail servers across the internet until it reaches the recipient's mail server. It operates on ports 25 (server-to-server), 587 (submission with authentication), or 465 (SMTPS).

2. POP3 (Post Office Protocol v3)

Role: Used for **retrieving** emails from a mail server to a local client. POP3 typically downloads messages to the user's device and **deletes them from the server** (though "leave on server" options exist). It is suitable for users who access email from a single device and want offline access. Operates on port 110 (or 995 for POP3S).

3. IMAP (Internet Message Access Protocol)

Role: The modern standard for **retrieving and synchronizing** email across multiple devices. Unlike POP3, IMAP keeps messages on the server, allowing users to access the same mailbox from phones, laptops, and web browsers while maintaining folder structures, read/unread status, and sent items consistently. Operates on port 143 (or 993 for IMAPS).

4. HTTP/HTTPS (Hypertext Transfer Protocol)

Role: Powers **web-based email services** such as Gmail, Outlook.com, and Yahoo Mail. Users access their mail through a web browser rather than a dedicated email client. HTTPS ensures that login credentials, message content, and attachments are encrypted during transit, preventing eavesdropping.

5. MIME (Multipurpose Internet Mail Extensions)

Role: An extension standard that enables emails to support **non-text content**. MIME defines how to encode attachments (images, documents, audio), multilingual character sets, and HTML formatting within the traditional text-based email format. Without MIME, emails would be limited to plain ASCII text.

6. TLS/SSL (Transport Layer Security / Secure Sockets Layer)

Role: Provides **encryption and security** for email communications. TLS is used to secure SMTP (SMTPS), POP3 (POP3S), and IMAP (IMAPS) connections, ensuring that messages and login credentials cannot be intercepted or tampered with during transmission between client and server.

Paper 2: CN 24-25 (Question Paper 5666)

Course: B.Sc.(H) Computer Science, Semester IV (NEP-UGCF-2022)

Q1. (b) What is the purpose of the following flag bits with respect to the TCP header? (3)

(i) SYN (ii) ECE (iii) URG (iv) FIN (v) DF

Answer:

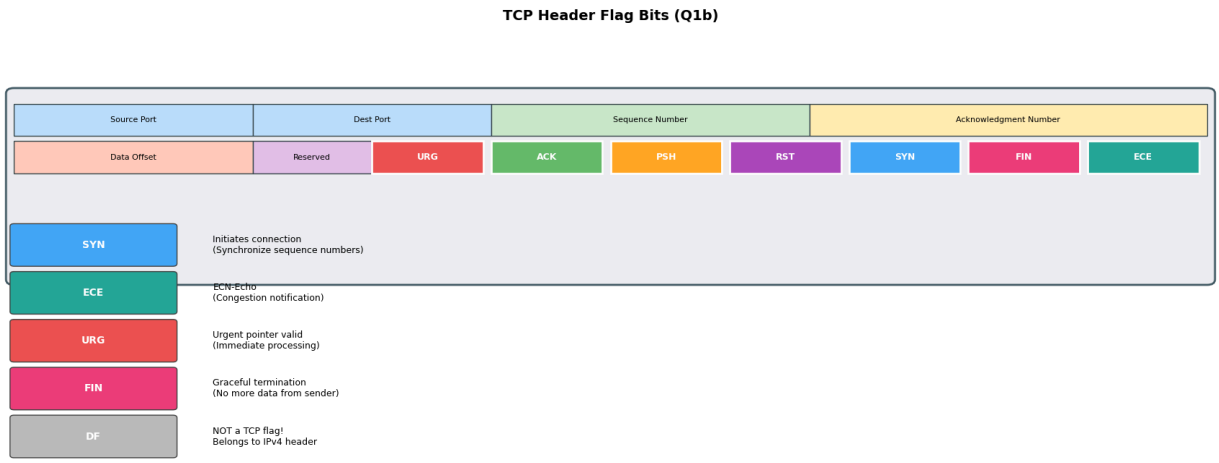

Figure 5: *TCP Header Flag Bits*

(i) SYN (Synchronize)

- Used to **initiate a TCP connection** between two hosts.
- It synchronizes the sequence numbers during the **three-way handshake** (SYN → SYN-ACK → ACK). The sender sets this flag to request the establishment of a new connection.

(ii) ECE (ECN-Echo)

- Stands for **Explicit Congestion Notification Echo**.
- During the handshake, it indicates that both hosts support **ECN**. After the connection is established, the receiver sets the ECE flag in ACK packets to inform the sender that a congestion-experienced (CE) packet was received, prompting the sender to reduce its transmission rate.

(iii) URG (Urgent)

- Indicates that the **Urgent Pointer field** in the TCP header contains a valid offset.
- When set, it tells the receiving application that the segment contains **urgent data** that should be processed immediately, bypassing normal buffering queues.

(iv) FIN (Finish)

- Used to **gracefully terminate a TCP connection** from one side.
- When a host has no more data to send, it sets the FIN flag to signal the end of its data stream. The connection is fully closed after a four-way exchange (FIN → ACK → FIN → ACK).

(v) DF (Don't Fragment)

- **This is NOT a TCP flag.** It belongs to the **IPv4 header** (in the 3-bit Flags field).
- When set to 1, it instructs routers **not to fragment** the IP packet. If the packet is too large for a network link, the router drops it and returns an ICMP "Fragmentation Needed" message instead. TCP has no DF flag.

Q1. (j) A URL has three components. Extract the three components of the given URL: http://www.abc.india.edu/index.html (3)

Answer:

The three components of the URL http://www.abc.india.edu/index.html are:

1. **Protocol (Scheme):** http:// - Specifies the protocol used to access the resource (Hypertext Transfer Protocol).
2. **Domain Name (Host):** www.abc.india.edu - Identifies the server where the resource is located.
3. **Path (Resource/File):** /index.html - Specifies the exact location and name of the file on the server.

Q6. (a) Compare and contrast TCP and UDP with respect to the following parameters: (7)

(a) Connection (b) Speed of data transfer (c) Header size (d) Reliability (e) Sequence of the segment received at the destination. Also, give the minimum and maximum size of a TCP segment.

Answer:

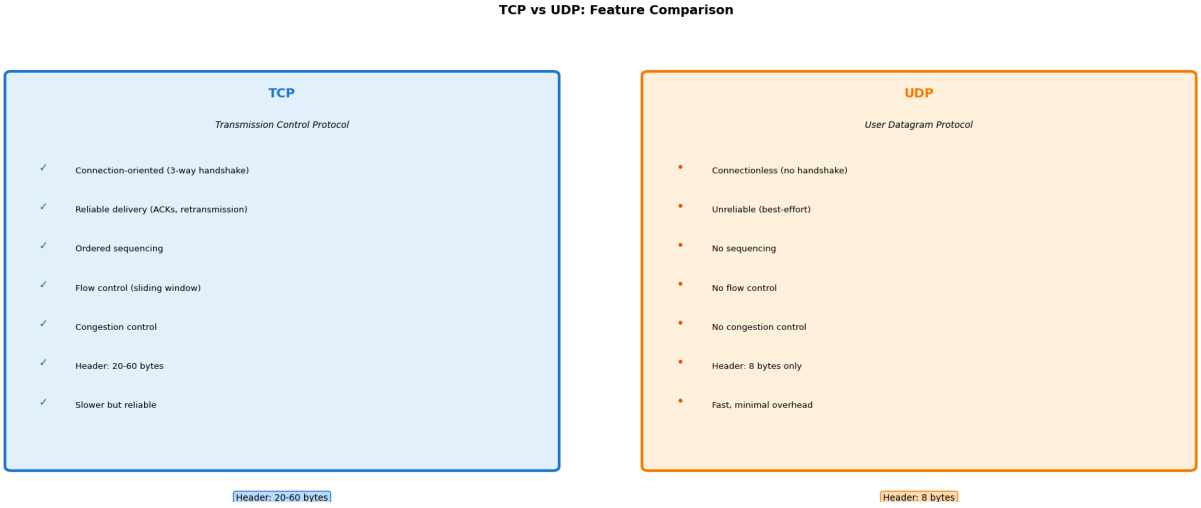

Figure 6: *TCP vs UDP: Feature Comparison*

Parameter	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
(a) Connection	**Connection-oriented.** It establishes a dedicated connection between sender and receiver via a three-way handshake (SYN, SYN-ACK, ACK) before data transfer begins.	**Connectionless.** No prior connection is established; datagrams are sent independently without any handshake.
(b) Speed of data transfer	**Slower.** The overhead of connection setup, acknowledgments, flow control, and error checking reduces transmission speed.	**Faster.** The absence of connection setup, ACKs, and retransmission mechanisms allows data to be sent with minimal delay.
(c) Header size	**Variable: 20 to 60 bytes.** The standard header is 20 bytes, but the Options field can extend it up to 60 bytes.	**Fixed: 8 bytes.** The UDP header contains only Source Port, Destination Port, Length, and Checksum fields.
(d) Reliability	**Highly reliable.** It guarantees delivery through acknowledgments (ACKs), sequence numbers, retransmission of lost segments, and error checking via checksum.	**Unreliable (best-effort).** It provides no guarantees for delivery, duplication control, or error recovery beyond a basic checksum.
(e) Sequence of segments received	**Ordered delivery.** TCP assigns a sequence number to each segment; the receiver reassembles them in the correct order and requests retransmission if any are missing.	**No sequencing.** Datagrams may arrive out of order, duplicated, or not at all; the application layer must handle ordering if required.

Minimum and Maximum Size of a TCP Segment

- **Minimum size: 20 bytes -** This occurs when the segment contains only the TCP header with no options and no data payload.

- **Maximum size: 65,535 bytes** (theoretical) - This limit is imposed by the 16-bit Total Length field in the IP header. Consequently, the maximum TCP data payload is **65,515 bytes** when standard 20-byte IP and TCP headers are used. In practice, the Maximum Segment Size (MSS)-typically **1,460 bytes** over Ethernet-is negotiated during connection setup to avoid IP-level fragmentation.

Q6. (b) Discuss the various request methods provided by HTTP. (4)

Answer:

HTTP (Hypertext Transfer Protocol) defines several request methods that indicate the desired action to be performed on a specified resource. Here are the primary methods:

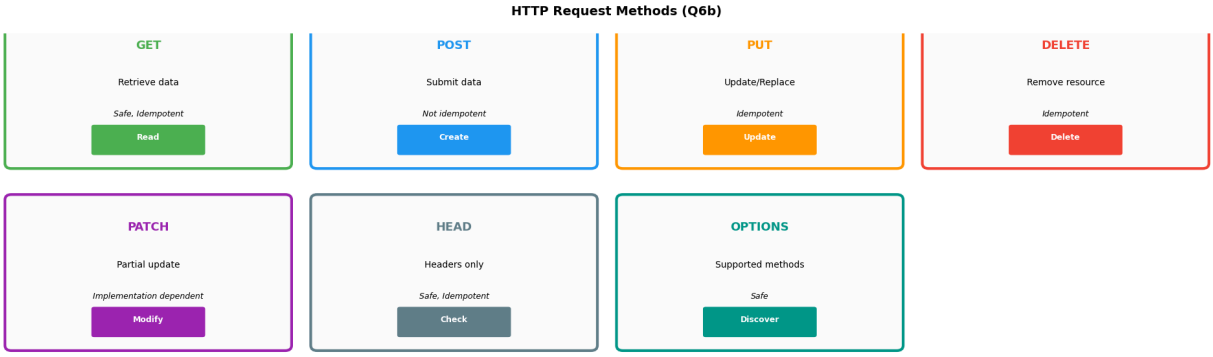

Figure 7: *HTTP Request Methods*

1. GET

- **Purpose:** Retrieves data from a specified resource.
- **Characteristics:** Safe and idempotent-multiple identical requests produce the same result without side effects.
- **Usage:** Loading web pages, searching, filtering data. Parameters are appended to the URL (query string), making them visible and bookmarkable.

2. POST

- **Purpose:** Submits data to be processed to a specified resource.
- **Characteristics:** Not idempotent-submitting the same data multiple times may create duplicate entries.
- **Usage:** Form submissions, file uploads, creating new records in a database. Data is sent in the request body, keeping it hidden from the URL.

3. PUT

- **Purpose:** Updates or replaces an existing resource entirely with the submitted data.
- **Characteristics:** Idempotent-repeating the same PUT request yields the same result.
- **Usage:** Updating user profiles, replacing document contents.

4. DELETE

- **Purpose:** Removes the specified resource from the server.
- **Characteristics:** Idempotent-deleting a resource multiple times has the same effect as deleting it once.
- **Usage:** Removing accounts, blog posts, or files from a database or storage system.

5. PATCH

- **Purpose:** Applies partial modifications to a resource (unlike PUT, which requires the complete resource).
- **Characteristics:** More bandwidth-efficient than PUT for minor updates; idempotency depends on implementation.
- **Usage:** Updating a single field in a user profile, such as changing just the email address.

6. Additional Methods

- **HEAD:** Identical to GET but returns only headers, not the body-useful for checking resource existence or metadata without downloading content.
- **OPTIONS:** Returns the HTTP methods supported by the server for a specific URL, often used in CORS preflight requests.
- **TRACE:** Echoes back the received request for diagnostic purposes (rarely used due to security risks).
- **CONNECT:** Establishes a tunnel to the server, commonly used for HTTPS connections through proxy servers.

Q6. (c) Ravi writes an email to his friend and clicks the "Send" button. During this process, a protocol is responsible for transferring Ravi's message from his mail server to the recipient's mail server over the internet. Which protocol is involved in this transfer, and how does it facilitate communication between the mail servers? (4)

Answer:

The protocol responsible for transferring Ravi's email from his mail server to the recipient's mail server is **SMTP (Simple Mail Transfer Protocol)**.

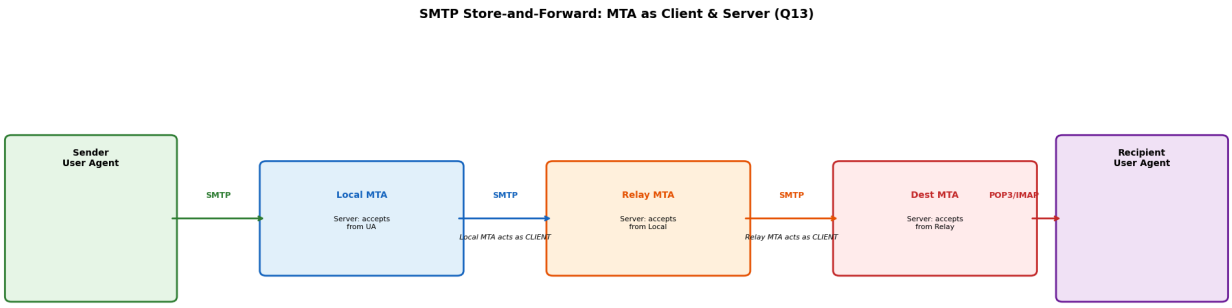

Figure 8: *SMTP Store-and-Forward: MTA as Client & Server*

How SMTP Facilitates Server-to-Server Communication:

1. Push-Based Transfer Mechanism

SMTP operates as a "push" protocol. Once Ravi clicks send, his outgoing mail server (SMTP client) actively initiates a connection to the recipient's incoming mail server (SMTP server) to deliver the message, rather than waiting for the recipient to fetch it.

2. TCP Connection Establishment

The sending mail server establishes a reliable TCP connection (typically over port 25 or 587) with the destination mail server. This ensures the entire email content-including headers, body, and attachments-is transmitted without corruption or loss.

3. Store-and-Forward Process

If the recipient's server is unavailable, SMTP employs a store-and-forward mechanism. The sending server queues the message and retries delivery periodically (usually over several days) until the destination server accepts it or the message times out.

4. Command-Response Dialogue

The servers communicate through a structured text-based exchange:

- The sending server identifies itself using EHLO or HELO
- Specifies the sender with MAIL FROM:
- Identifies the recipient with RCPT TO:
- Transfers the actual message content via the DATA command
- Terminates the session with QUIT

5. Multi-Hop Relay

If the recipient's server is not directly reachable, intermediate mail servers can relay the message hop-by-hop until final delivery, with each server using SMTP to pass the message to the next.

Once the email reaches the recipient's mail server, it remains stored there until the recipient retrieves it using protocols like **POP3** or **IMAP**.

Paper 3: CN 23-24 (Question Paper 4133)

Q2. (a) Compare and contrast TCP and UDP (User Datagram Protocol) in terms of their features, advantages, and disadvantages. Discuss scenarios where each protocol would be suitable for data transmission. (7)

Answer:

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are both core transport layer protocols in the TCP/IP suite, but they serve fundamentally different purposes and are optimized for different types of network communication.

Core Features Comparison

Feature	TCP	UDP
Connection	Connection-oriented (requires three-way handshake)	Connectionless (no handshake)
Reliability	Guaranteed delivery with acknowledgments (ACKs)	No guaranteed delivery
Ordering	Sequencing ensures data arrives in order	No sequencing; packets may arrive out of order
Error Checking	Extensive (checksums, retransmission)	Basic checksum only
Flow Control	Yes (sliding window mechanism)	No
Congestion Control	Yes (reduces rate when network is congested)	No
Overhead	High (20-60 byte header)	Low (8 byte header)
Speed	Slower due to setup and verification	Fast, minimal delay

Advantages and Disadvantages

TCP

Advantages:

- **Reliability:** Ensures all data reaches its destination without corruption or loss
- **Ordered delivery:** Reassembles packets in the correct sequence, preventing data jumbling
- **Error recovery:** Automatically retransmits lost or corrupted packets
- **Flow control:** Prevents overwhelming the receiver with too much data at once
- **Congestion avoidance:** Adjusts transmission rates to maintain network stability

Disadvantages:

- **Latency:** Three-way handshake adds initial delay
- **Overhead:** Large header size and acknowledgment traffic consume bandwidth
- **Slower transmission:** Error checking and ordering mechanisms reduce throughput
- **Resource intensive:** Requires more memory and processing power to maintain connection state

UDP

Advantages:

- **Low latency:** No connection setup; data transmission begins immediately
- **Minimal overhead:** Small header size leaves more room for payload data
- **High speed:** No waiting for acknowledgments or retransmissions
- **Multicast/Broadcast support:** Efficiently sends data to multiple recipients simultaneously
- **Simplicity:** Easier to implement in applications where reliability isn't critical

Disadvantages:

- **Unreliable:** No guarantee that packets will arrive, or arrive only once
- **No ordering:** Applications must handle sequence management if needed
- **No congestion control:** Can contribute to network congestion and packet loss
- **No flow control:** Risk of overwhelming the receiver

Suitable Scenarios

When to Use TCP

TCP is ideal when **data integrity and completeness** are more important than speed:

- **Web browsing (HTTP/HTTPS):** Web pages must load completely and correctly
- **Email (SMTP, POP3, IMAP):** Message content cannot be corrupted or lost
- **File transfers (FTP, SFTP):** Files must arrive intact without missing data
- **Database transactions:** Financial records and user data require accuracy

- **Remote administration (SSH, Telnet):** Command execution must be precise and ordered

When to Use UDP

UDP excels when **speed and real-time delivery** matter more than absolute reliability:

- **Video/audio streaming (Netflix, Spotify, YouTube):** Occasional pixelation is preferable to buffering delays
- **Voice over IP (VoIP) and video conferencing (Zoom, Skype):** Real-time conversation cannot tolerate retransmission delays
- **Online gaming:** Player positions and actions must update immediately
- **DNS queries:** Fast lookup is essential; if a query fails, it can simply be retried
- **Live broadcasts and multicast:** Efficiently distributes data to thousands of simultaneous viewers
- **IoT sensor data:** Frequent small readings where missing occasional packets is acceptable

From the Book

Transport Layer Foundations (Section 6.1.1)

Q1. Process-to-Process vs. Host-to-Host Delivery

The network layer provides host-to-host delivery, yet the transport layer is required for process-to-process delivery. Explain why the transport layer is necessary, and describe how it uses **TSAPs (Transport Service Access Points)** to distinguish between multiple processes running on the same host. (Reference: Section 6.1.1, p. 496-497)

Answer:

Figure 9: *TSAPs (Ports): Well-Known Server Ports vs Ephemeral Client Ports*

The **network layer** provides host-to-host delivery: it is responsible for getting an IP packet from the source host to the destination host (i.e., from one NSAP to another NSAP, where an NSAP is a Network Service Access Point such as an IP address). However, once the packet arrives at the destination machine, the operating system still needs to know *which process* should receive the data. That is the job of the **transport layer**.

To distinguish among the many processes that may be running simultaneously on a single host, the transport layer defines endpoints called **TSAPs (Transport Service Access Points)**. In the Internet, these endpoints are known as **ports**. A TSAP is a specific endpoint in the transport layer, whereas an NSAP (e.g., an IP address) is an endpoint in the network layer. Because a single host has only one NSAP (or one IP address) but may be running many application processes, TSAPs are necessary to multiplex and demultiplex traffic so that each segment is delivered to the correct process.

Q2. Client-Server Roles at the Transport Layer

In the transport-layer client-server model, what are the distinct responsibilities of the **client TSAP** and the **server TSAP**? Why must the server's TSAP address be well-known in advance, while the client's TSAP address is typically chosen dynamically? (Reference: Section 6.1.1, p. 497-498)

Answer:

In Tanenbaum's TSAP model, a transport connection is established between a **local TSAP** on the client host and a **remote TSAP** on the server host.

Server TSAP:

- A server process *attaches itself* to a specific TSAP (e.g., TSAP 1522) and listens for incoming connection requests.
- Server TSAP addresses are **stable and well-known**. They are permanently associated with standard services and listed in system files (e.g., /etc/services on UNIX). For example, the mail server listens on TCP port 25.
- They must be well-known in advance because a client cannot issue a CONNECT request unless it already knows the destination TSAP address. As the textbook asks: "How does the user process on host 1 know that the mail server is attached to TSAP 1522?" The answer is that these addresses are learned from stable directories or long-standing convention.

Client TSAP:

- A client process *attaches itself* to an arbitrary local TSAP (e.g., TSAP 1208) purely for the purpose of initiating a connection, and then issues a CONNECT request naming its own TSAP as the source and the server's well-known TSAP as the destination.
- Client TSAPs are **chosen dynamically** because user processes, in general, do not have TSAP addresses that are known in advance, and they may exist for only a short time. The client's endpoint is temporary; it is allocated just long enough to establish the connection, exchange data, and release the connection.

Why the asymmetry?

The server must be reachable by strangers, so its TSAP address must be published and stable. The client, by contrast, is the initiator; it only needs a temporary return address so the server knows where to send responses. As the text notes, "user processes, in general, often want to talk to other user processes that do not have TSAP addresses that are known in advance."

Connectionless Client-Server - UDP (Section 6.4 up to Section 6.4.1)

Q3. UDP's "Minimal" Service Model

Tanenbaum states that UDP "does almost nothing beyond sending packets between applications." List the specific pieces of information carried in the **8-byte UDP header**, and identify which field is optional in IPv4 but mandatory in IPv6. (Reference: Section 6.4.1, p. 541-543)

Answer:

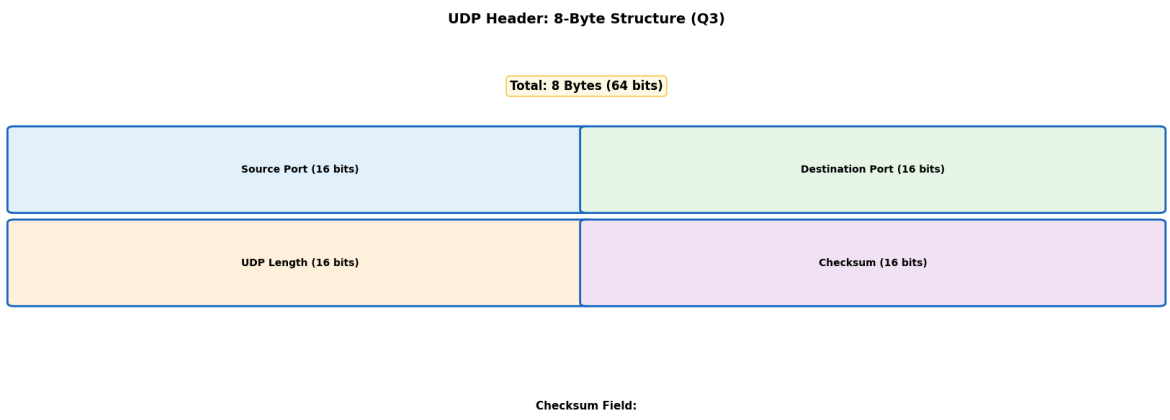

Figure 10: *UDP Header: 8-Byte Structure*

The **8-byte UDP header** contains the following four fields:

Field	Size	Purpose
Source port	16 bits	Identifies the sending process (ephemeral on the client)
Destination port	16 bits	Identifies the receiving process on the destination host
UDP length	16 bits	Length of the UDP datagram (header + data) in bytes
Checksum	16 bits	Optional error-detection over the header, data, and a conceptual IP pseudoheader

The **Checksum** is the field that is **optional in IPv4** but **mandatory in IPv6**. The textbook explicitly notes that the UDP checksum is optional: "An optional Checksum is also provided for extra reliability... If the checksum is not computed, it is stored as a 0." However, in IPv6 the checksum is mandatory because IPv6

has no header checksum of its own (unlike IPv4), leaving UDP with the sole responsibility for end-to-end integrity checking.

Q4. Connectionless Communication

Explain why UDP is classified as **connectionless**. In the context of client-server communication, describe what happens when a UDP client sends a segment to a server that is not currently running-does the network store the segment, drop it, or notify the sender? What specific role does UDP *not* perform that TCP would in this scenario? (Reference: Section 6.4, p. 541; Section 6.4.1, p. 542)

Answer:

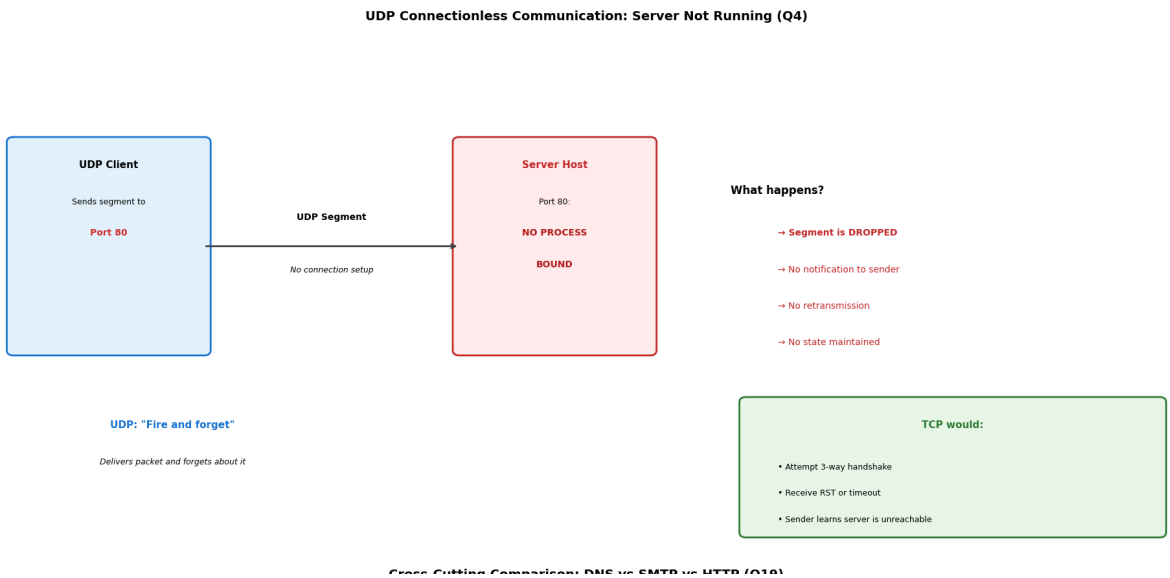

Figure 11: *UDP Connectionless Communication: Server Not Running*

Why UDP is classified as connectionless:

UDP requires **no connection setup** before data transmission. As Tanenbaum states, UDP "does not do flow control, congestion control, or retransmission upon receipt of a bad segment" and "does almost nothing beyond sending packets between applications." A UDP client simply formats a segment and hands it to IP for delivery; there is no handshake, no state maintenance, and no release procedure.

What happens when a UDP segment is sent to a server that is not running:

The segment is **dropped**. Because UDP itself does not maintain any connection state, it has no mechanism to hold, queue, or retransmit the segment. When the segment arrives at the destination host, the transport layer checks the destination port number; if no process has bound to that port (i.e., the server is not running), the OS has no recipient to deliver it to, so the segment is discarded.

What UDP does NOT do that TCP would:

UDP does **not notify the sender** that the delivery failed. The sending client has no automatic signal-no RST segment, no timeout from a connection-establishment timer, and no retransmission-to tell it that the server is

unreachable. In contrast, TCP would attempt a **three-way handshake**; if the server process is not listening, the client's SYN would either receive a **RST (reset)** segment in reply or simply time out, thereby informing the client that the connection cannot be established.

In short: UDP delivers the packet and forgets about it; TCP validates reachability before data transfer and reports failure if the server is absent.

Q5. Port Numbers and Demultiplexing

A UDP segment arrives at a host with destination IP address X and destination port 80. How does the transport layer know which application process should receive this segment? What would happen if two different client applications on the same host both happened to choose the same ephemeral source port number to contact the same web server? (Reference: Section 6.4.1, p. 542-543)

Answer:

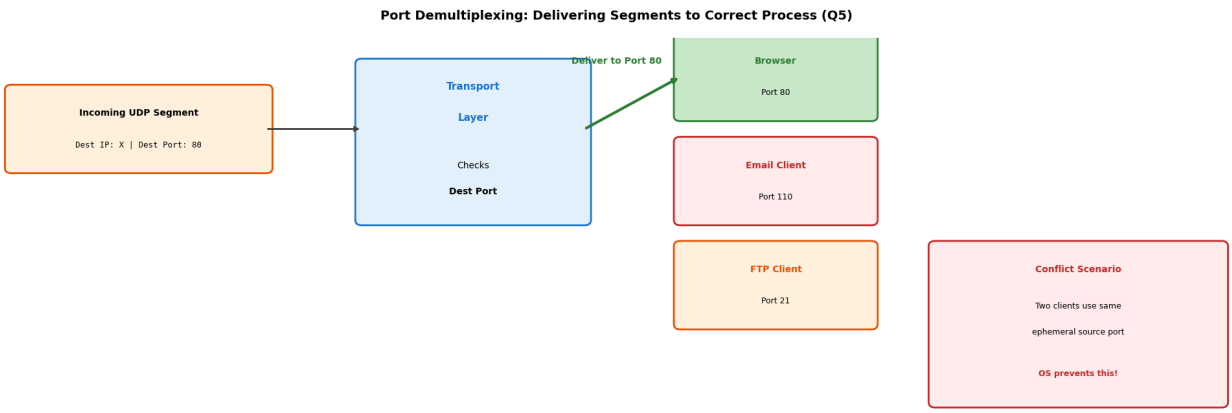

Figure 12: *Port Demultiplexing: Delivering Segments to Correct Process*

How the transport layer knows which process should receive the segment:

When a UDP segment arrives at a host, the transport layer examines the **destination port** field in the UDP header and delivers the payload to the process **attached to that port**. This attachment is established in advance when the application uses the **BIND** primitive (or something similar) to rent the port, like a mailbox. Because port numbers identify endpoints *within* each machine, the destination port acts as the demultiplexing key that tells the OS which application process should receive the incoming packet.

What happens if two clients choose the same ephemeral source port:

The textbook describes the source port as being "primarily needed when a reply must be sent back to the source." The server copies the source port from the incoming segment into the destination port of its reply so the response reaches the correct client process. If two different client applications on the *same host* both

happened to choose the *same* ephemeral source port to contact the *same* web server (same destination IP and port), the replies returning from that server would carry identical destination port numbers. Because ports identify endpoints *within* a machine, the OS would have no way to distinguish which of the two client processes should receive which reply-the demultiplexing would be ambiguous. In practice, the operating system prevents this by allocating **unique ephemeral ports** to each outgoing connection so that no two local processes share the same source port for traffic to the same destination.

Connection-Oriented Client-Server - TCP (Section 6.5.3 - Section 6.5.4)

Q6. TCP Protocol Fundamentals

Contrast TCP with UDP by identifying the three major mechanisms TCP provides *on behalf of applications* that UDP does not. Then explain why Tanenbaum describes the relationship between the two protocols as "complementary." (Reference: Section 6.4 introductory text, p. 541; Section 6.5.3, p. 556)

Answer:

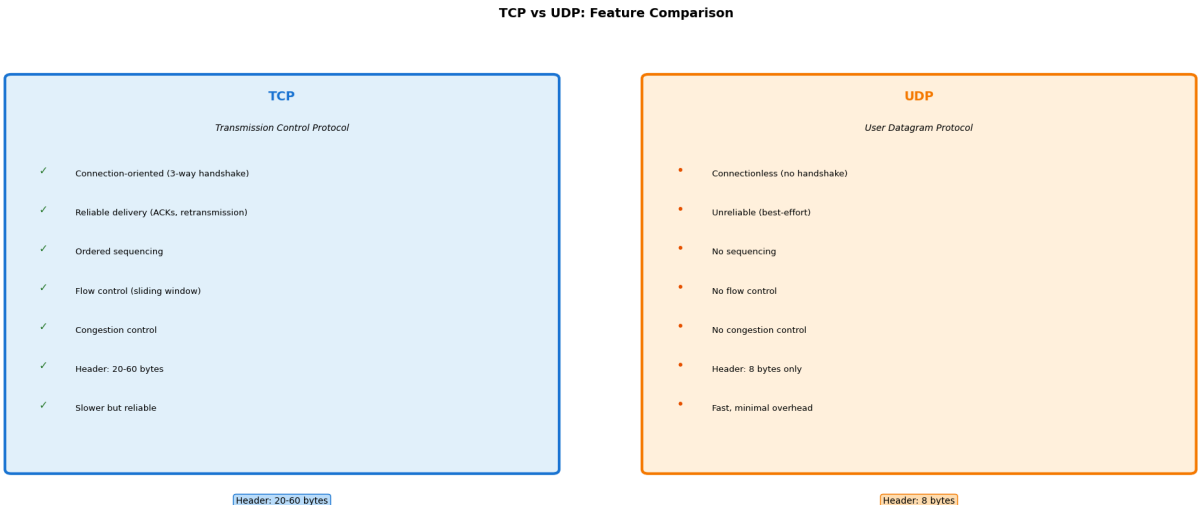

Figure 13: *TCP vs UDP: Complementary Protocols*

Three major mechanisms TCP provides that UDP does not:

Tanenbaum explicitly contrasts the two protocols at the start of Section 6.4:

1. **Reliability with retransmissions** - TCP detects lost or corrupted segments and retransmits them; UDP does not.
2. **Flow control** - TCP paces the sender so it does not overwhelm a slow receiver; UDP does not.
3. **Congestion control** - TCP throttles the sender to avoid overloading the network; UDP does not.

Why the relationship is described as "complementary":

The textbook states: "The Internet has two main protocols in the transport layer, a connectionless protocol and a connection-oriented one. The protocols complement each other."

- **UDP is minimal:** "It does almost nothing beyond sending packets between applications, letting applications build their own protocols on top as needed." It imposes no policy, so applications that need speed (e.g., DNS queries, streaming media) are not delayed by setup or retransmission overhead.
- **TCP is maximal:** "It does almost everything. It makes connections and adds reliability with retransmissions, along with flow control and congestion control, all on behalf of the applications that use it." Applications that need reliable delivery (e.g., file transfer, email, web browsing) get it automatically without having to implement it themselves.

Because different applications have fundamentally different needs-some require speed over reliability, others require guaranteed delivery over latency-having *both* a do-nothing protocol and a do-everything protocol covers the full spectrum of application requirements. They are complementary because each fills the gaps the other leaves open.

Q7. The TCP Segment Header - Field Functions

The fixed TCP header is **20 bytes** (without options). For each of the following fields, state its purpose in the client-server communication model: Sequence Number, Acknowledgment Number, Window Size, Checksum, URG pointer (and how it differs from the urgent flag). (Reference: Section 6.5.4, p. 557-559)

Answer:

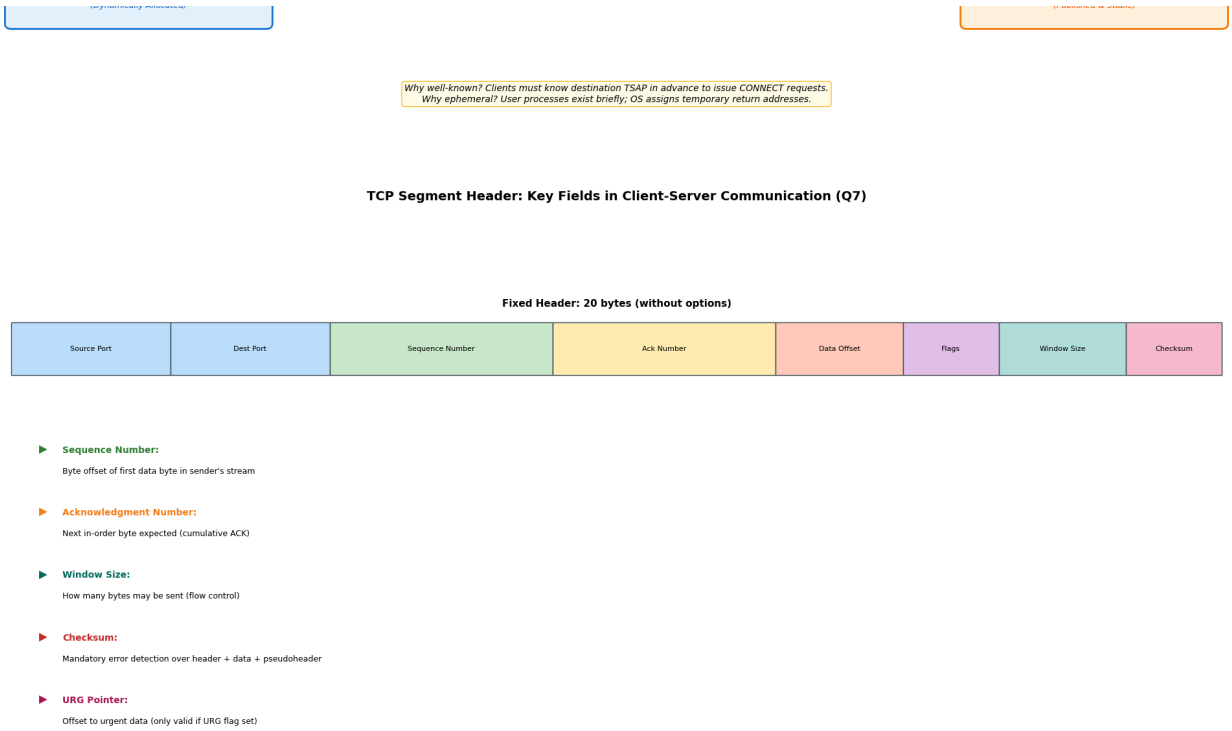

Figure 14: *TCP Segment Header: Key Fields in Client-Server Communication*

The fixed TCP header is **20 bytes** (without options). Each field performs a specific function in the client-server model:

Sequence Number	Identifies the byte offset of the first data byte in the segment within the sender's byte stream. Because every byte in a TCP stream is numbered, this field tells the receiver where the data belong in the overall stream.
Acknowledgment Number	Specifies the **next in-order byte expected** by the receiver, not the last byte correctly received. It is a **cumulative acknowledgment** because it summarizes all received data up to that point with a single number.
Window Size	Tells the sender how many bytes may be sent starting from the byte acknowledged. It implements the variable-sized sliding window for flow control. A value of 0 means: "I have received bytes up through k but I do not want any more just now, thank you."
Checksum	Provides extra reliability by checksuming the header, the data, and a conceptual pseudoheader (exactly like UDP, but with protocol number 6 for TCP and the checksum is **mandatory**).
URG pointer	Used only when the **URG** flag is set to 1. It indicates a byte offset from the current sequence number at which urgent data are to be found. This is a bare-bones way of allowing the sender to signal the receiver without getting TCP itself involved in the reason for the interrupt.

Important distinction: The URG flag and the URG pointer are not the same thing. The **URG flag** (1 bit) merely signals whether the Urgent pointer field is in use. The **Urgent pointer** itself (16 bits) contains the actual byte offset. This facility is rarely used in practice.

Q8. TCP Flags and Connection State

The 6-bit **Flags** field in the TCP header contains bits including **SYN, FIN, RST, PSH, ACK,** and **URG**. Describe the exact sequence of these flags exchanged during the **three-way handshake** that establishes a TCP connection between a client and server. Which flags are set in the first segment sent by the client? (Reference: Section 6.5.4, p. 558; Section 6.5.5, p. 560)

Answer:

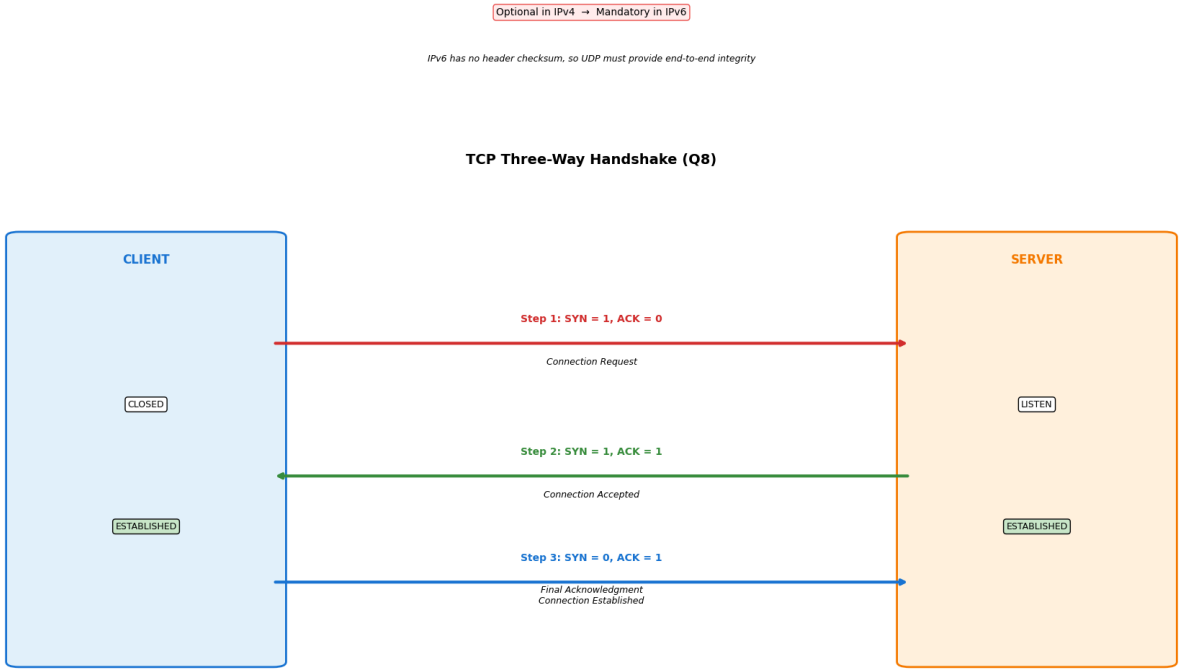

Figure 15: *TCP Three-Way Handshake*

The TCP header contains **eight 1-bit flags**. The normal **three-way handshake** that establishes a TCP connection proceeds as follows:

Step	Sender	Flags	Meaning
1	Client → Server	**SYN = 1, ACK = 0**	Connection request. The piggyback acknowledgement field is not in use.
2	Server → Client	**SYN = 1, ACK = 1**	Connection accepted. The server acknowledges the client's SYN and sends its own SYN.
3	Client → Server	**SYN = 0, ACK = 1**	Final acknowledgment of the server's SYN. The connection is now established.

Flags set in the first segment sent by the client:

The client's initial segment carries **SYN = 1** and **ACK = 0**. As the textbook explains: "The connection request has SYN=1 and ACK=0 to indicate that the piggyback acknowledgement field is not in use." Because the client is initiating the connection, it has no prior data from the server to acknowledge, so the ACK bit is off.

How SYN distinguishes between request and accept:

The SYN bit is used to denote both **CONNECTION REQUEST** and **CONNECTION ACCEPTED**, with the **ACK bit** used to distinguish between the two. The request has ACK=0; the acceptance has ACK=1.

A DNS query fits into a single UDP datagram, but if DNS were forced to run over TCP, it would require multiple segments just for the query and response. Calculate the minimum **header overhead** (in bytes) for a single DNS query using (a) UDP, and (b) TCP (assuming no TCP options). Why does this difference matter for request-response applications with small payloads? (Reference: Section 6.4.1, p. 542; Section 6.5.4, p. 557)

Answer:

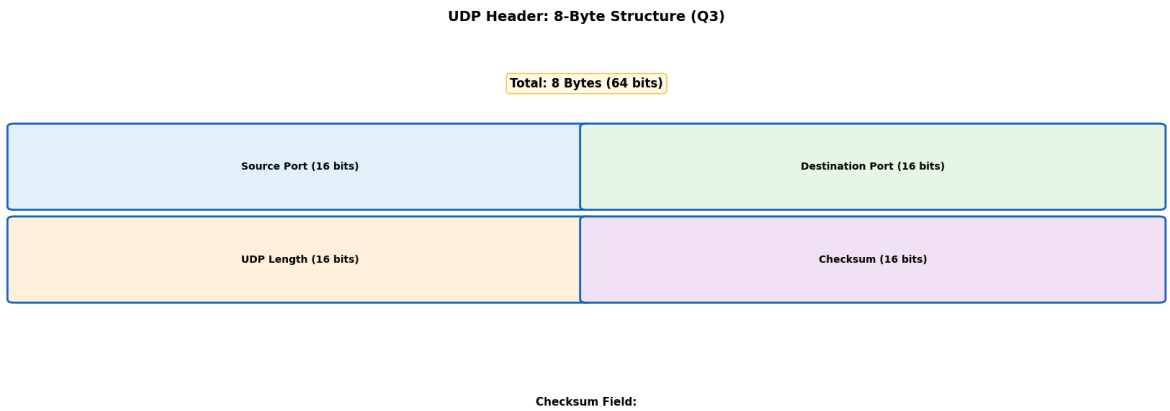

Figure 16: *UDP (8 bytes) vs TCP (20 bytes) Header Overhead Comparison*

(a) UDP header overhead:

The UDP header is **8 bytes** (four 16-bit fields: Source port, Destination port, UDP Length, Checksum).

(b) TCP header overhead:

The fixed TCP header is **20 bytes** (without options).

Why this difference matters for request-response applications with small payloads:

DNS queries and responses are typically small enough to fit into a single packet. With UDP, the protocol overhead is only 8 bytes, making the datagram efficient to transmit and process. With TCP, the same query would carry 20 bytes of header overhead-**2.5x the header size**-plus the additional segments required just to establish the connection (SYN, SYN-ACK, ACK) before any data can be exchanged. For an application like DNS, where the payload is often smaller than the TCP header itself, this overhead is disproportionately large. UDP's minimal header and connectionless design make it the natural choice for lightweight, request-response transactions where speed and efficiency outweigh the need for connection setup, flow control, and retransmission.

Application-Layer Client-Server Examples

A. DNS - Domain Name System (Section 7.1)

Q10. DNS Client-Server Architecture

When a browser needs to resolve www.cs.washington.edu, it calls a **resolver** rather than directly querying a root name server. Trace the chain of client-server interactions: from the resolver on the client machine, to the **local DNS server**, and then to the hierarchy of name servers. Why are the query and response messages sent as **UDP packets** rather than TCP segments? (Reference: Section 7.1, p. 611-612)

Answer:

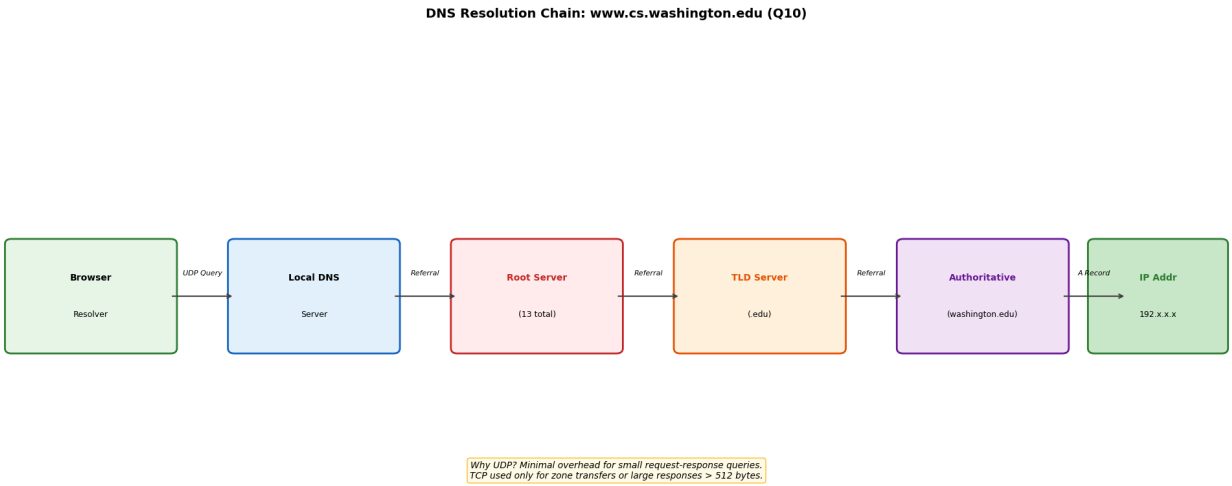

Figure 17: *DNS Resolution Chain: www.cs.washington.edu*

Chain of client-server interactions for resolving www.cs.washington.edu:

1. **The Resolver (client-side library):** The browser does not query a root server directly. Instead, it calls a **library procedure called the resolver** (e.g., gethostbyname), passing the host name as a parameter. The resolver is the client-side stub that initiates the DNS lookup on behalf of the application.
2. **Local DNS Server:** The resolver sends the query to a **local DNS server** (often configured by the ISP or local network administrator). This server acts as the first point of contact.
3. **Hierarchy of name servers (if the local server cannot resolve from cache):**
   - **Root Name Server:** The local name server contacts one of the **13 root DNS servers** (a.root-servers.net through m.root-servers.net). The root server does not know the final address, but it knows the name server for the .edu top-level domain and returns that referral.
   - **Top-Level Domain (TLD) Name Server:** The local server then queries the .edu name server, which returns the address of the authoritative name server for the washington.edu domain.

- **Authoritative Name Server:** The local server queries the UW name server, which may in turn refer it to the cs.washington.edu zone name server (if CS runs its own zone). Finally, the authoritative server returns the **A record** containing the IP address.

4. **Response to resolver:** The local DNS server returns the IP address to the resolver, which passes it back to the calling application. Only then can the application establish a connection to the target host.

Why UDP is used:

The textbook states plainly: "The query and response messages are sent as UDP packets." UDP is used because DNS resolution is a **small, single-packet request-response exchange** where the overhead of establishing a TCP connection (three-way handshake, 20-byte header, teardown) would be excessive relative to the tiny amount of data being transferred. A UDP query fits in one datagram and receives a UDP response, making the lookup fast and lightweight. Once the IP address is obtained, the application can then establish a **TCP connection** with the target host if needed (e.g., for HTTP or SMTP), but the DNS lookup itself benefits from UDP's minimalism.

Q11. Zones and Distributed Authority

Tanenbaum notes that a single name server containing the entire DNS database would be "so overloaded as to be useless." Explain how the DNS namespace is divided into **zones**, and identify the three tiers of the DNS server hierarchy that handle queries when a local name server cannot resolve a name from its cache. (Reference: Section 7.1.3, p. 619-621)

Answer:

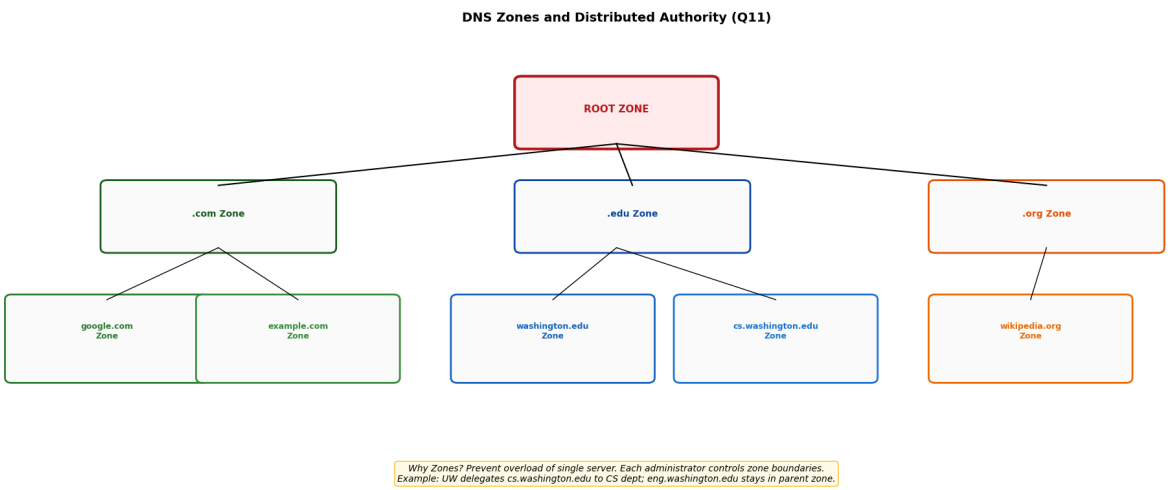

Figure 18: *DNS Zones and Distributed Authority*

Why the DNS namespace is divided into zones:

Tanenbaum states: "In theory at least, a single name server could contain the entire DNS database and respond to all queries about it. In practice, this server would be so overloaded as to be useless. Furthermore, if it ever went down, the entire Internet would be crippled." To avoid both overload and catastrophic single points of failure, the DNS name space is partitioned into **nonoverlapping zones**, each representing a contiguous portion of the domain tree. Where the zone boundaries are placed is up to the zone administrator (e.g., the University of Washington delegates cs.washington.edu as a separate zone because the Computer Science department chooses to run its own name server, while eng.washington.edu remains within the larger washington.edu zone).

Three tiers of the DNS server hierarchy:

When a local name server cannot resolve a name from its cache, it performs an **iterative query** sequence through three tiers:

1. **Root Name Servers** - The local server first contacts one of the 13 root servers (a.root-servers.net through m.root-servers.net). The root server does not know the final address, but it returns the name and IP address of the **TLD name server** for the relevant top-level domain.
2. **Top-Level Domain (TLD) Name Server** - The local server queries the TLD server (e.g., a.edu-servers.net for .edu). This server returns the name and IP address of the **authoritative name server** for the specific domain.
3. **Authoritative Name Server** - The local server finally queries the authoritative server for the target zone (e.g., the UW Computer Science name server for cs.washington.edu). This server is authoritative for that domain and returns the definitive resource record.

B. Message Transfer - SMTP (Section 7.2.4)

Q12. SMTP's Two Uses in Email Architecture

SMTP is used for two distinct steps in email architecture: **mail submission** and **message relay**. Distinguish these two uses in terms of *which* client talks to *which* server, and identify the protocol layer on which SMTP runs. Why was SMTP originally designed to establish a transport connection directly from the source machine to the destination machine? (Reference: Section 7.2.4, p. 637-638)

Answer:

Use	Step	Client	Server	Purpose
Mail Submission	Step 1	User Agent	Local Mail Server (MTA)	The means by which user agents send messages *into* the mail system for delivery.

Message Transfer (Relay)	Step 2	Sending MTA	Receiving MTA	Transfers messages between message transfer agents, delivering mail from the sending MTA to the receiving MTA in one hop.

Protocol layer:

SMTP runs over **TCP** (port 25).

Why SMTP was originally designed for direct source-to-destination delivery:

The textbook explains: "The simplest way to move messages is to establish a transport connection from the source machine to the destination machine and then just transfer the message. This is how SMTP originally worked." The rationale was simplicity-if the sender knows the recipient's host, opening a direct TCP connection and pushing the message across requires no intermediate store-and-forward logic, no queuing at relay hosts, and no extra hops. This direct model minimizes latency and implementation complexity for the common case where both hosts are online. Over time, however, the architecture evolved to support mail submission as a separate function so that user agents could hand off messages to local servers, which then handle the relay and retry logic on their behalf.

Q13. Store-and-Forward vs. Direct Delivery

Modern email often does not travel directly from the sender's host to the recipient's host. Explain how **Message Transfer Agents (MTAs)** act as both clients and servers in a store-and-forward chain, and why this architecture is necessary even though the sender and recipient may both be online simultaneously. (Reference: Section 7.2.4, p. 637-638; Fig. 7-7)

Answer:

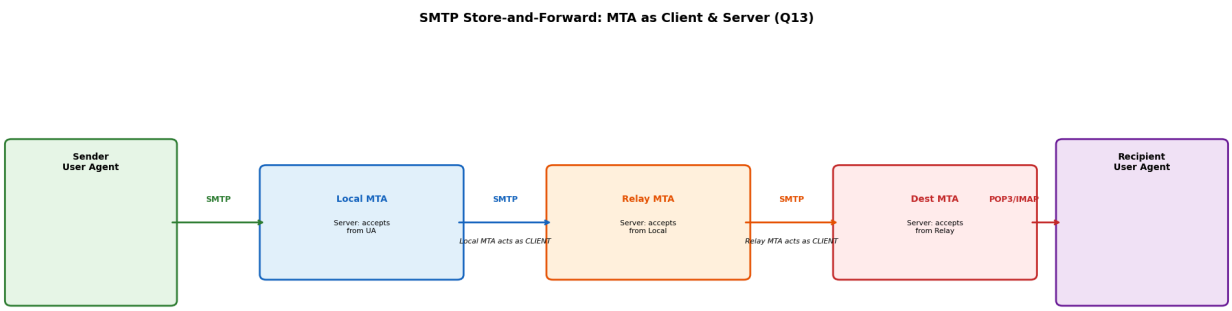

Figure 19: *SMTP Store-and-Forward: MTA as Client & Server*

How MTAs act as both clients and servers:

Message Transfer Agents (MTAs) operate in a **store-and-forward chain**. Consider the email architecture: the sender's user agent submits the message to its **local mail server** (MTA) using SMTP as a client. That

local MTA then becomes an SMTP **client** itself when it forwards the message to the next MTA in the chain (step 2). The receiving MTA, in turn, acts as an SMTP **server** accepting the connection. This pattern repeats until the message reaches the recipient's local mail server, which holds the message for final delivery. Thus, every intermediate MTA is simultaneously a server (accepting mail from upstream) and a client (relaying mail downstream).

Why this architecture is necessary even when both parties are online:

The textbook explains that SMTP was originally designed to establish a direct transport connection from the source machine to the destination machine. However, the modern email architecture decouples the sender from the recipient for several practical reasons:

1. **Reliability:** If the destination host is temporarily unreachable, the local MTA can store the message and retry later-something the sender's user agent is not designed to do.
2. **Policy and security:** Organizations route all outgoing mail through a central mail server to enforce spam filtering, virus scanning, and logging.
3. **Addressing convenience:** The sender does not need to know the exact host where the recipient is currently logged in; the recipient's local mail server handles final delivery.
4. **Asynchronous communication:** Even if both users are online, their machines may not be running mail servers, may be behind firewalls, or may have intermittent connectivity. The store-and-forward model ensures delivery without requiring both endpoints to be simultaneously reachable.

C. The World Wide Web & Architectural Overview (Section 7.3 - Section 7.3.1, up to p. 652)

Q14. Web Components and Their Roles

Using the architectural components shown in Fig. 7-18 (p. 648), identify the client-side and server-side entities in a single web page fetch. Specifically: what is the role of the **browser**, what is the role of the **web server**, and what does Tanenbaum mean by stating that the user *"cannot tell"* whether linked content resides on the same machine or one halfway around the globe? (Reference: Section 7.3.1, p. 647-649)

Answer:

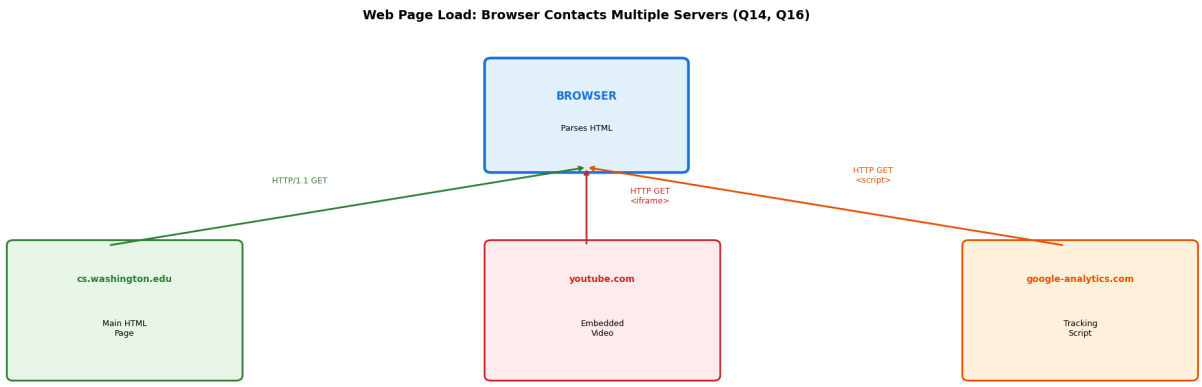

Figure 20: *Web Page Load: Browser Contacts Multiple Servers*

Roles of browser and web server (Fig. 7-18):

Component	Role
Browser (client-side)	A program that fetches the requested page, interprets its content (text, images, formatting commands, video), and displays it properly formatted on the screen. The browser also handles user interactions such as mouse clicks on hyperlinks, translating them into new fetch requests.
Web Server (server-side)	A server that listens for HTTP requests over TCP, locates the requested resource (either a static file on disk or a dynamic program/database query), and returns an HTTP response containing the page content.

"The user cannot tell":

Tanenbaum writes: "Every other page can be comprised of content on the same machine(s) as the first page, or on machines halfway around the globe. The user cannot tell. Page fetching is done by the browser, without any help from the user. Thus, moving between machines while viewing content is seamless."

This means the Web's architecture intentionally **hides the physical location of content**. When a user clicks a hyperlink, the browser automatically issues a new HTTP request to whatever server hosts the linked page-whether it is the same department server (cs.washington.edu) or a different server on another continent (youtube.com). The rendering and navigation experience is identical in both cases. This location transparency is a fundamental property of the Web: the URL naming system and the HTTP protocol abstract away geography, so users navigate by logical names rather than by physical machine locations.

Q15. Static vs. Dynamic Pages

A student accesses their university's course registration page and sees a personalized schedule. Another student accesses the university's academic calendar and sees the same PDF document every time. Using Tanenbaum's definitions, classify these two pages as **static** or **dynamic**, and explain what happens on the server side in each case. Why does the dynamic case require program execution at the server, while the static case does not? (Reference: Section 7.3.1, p. 648-649)

Answer:

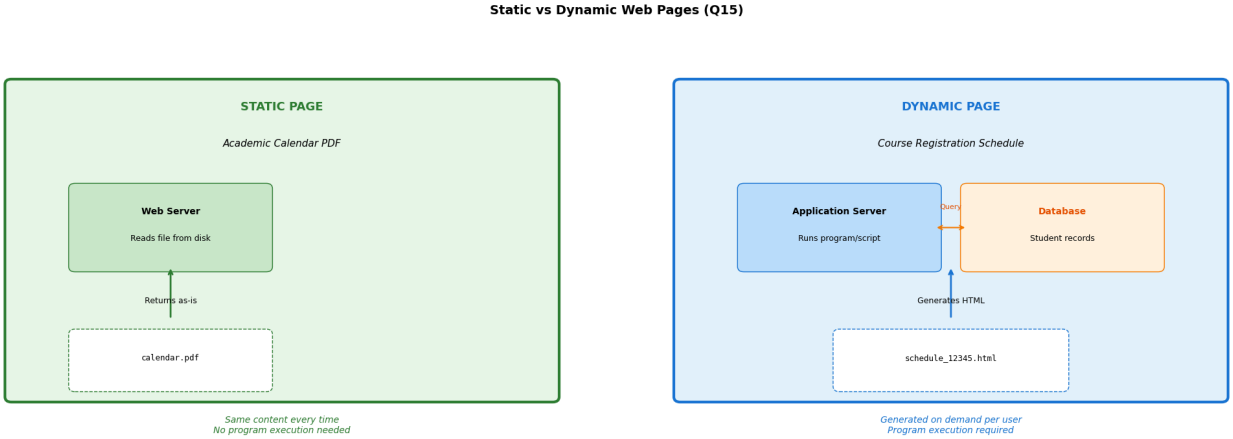

Figure 21: *Static vs Dynamic Web Pages*

Classification:

Page	Type	Reason
Academic calendar PDF	**Static**	It is "a document that is the same every time it is displayed."
Course registration page with personalized schedule	**Dynamic**	It is "generated on demand by a program" and "may present itself differently each time it is displayed."

What happens on the server side:

- **Static page:** The server simply **reads the document off disk** and returns it in the HTTP response. No program execution is required because the content is pre-authored and fixed.
- **Dynamic page:** The server **runs a program** (e.g., a CGI script, servlet, or database query) to generate the HTML on the fly. For the course registration example, the program would query the student database, construct a personalized schedule based on the logged-in user, and return the resulting page. As Tanenbaum notes, the dynamic case requires "program execution" because the page does not exist as a file until the request arrives.

Q16. Multiple Servers, Single Page

When loading a single web page (e.g., the UW CSE department homepage), the browser may contact **three different servers** to render the page. Give a concrete example from the textbook of content that might come from a different server than the main page, and explain how the browser integrates this content. What protocol is used for all these fetch operations? (Reference: Section 7.3.1, p. 648-649, Fig. 7-18)

Answer:

Concrete example from the textbook:

When loading the UW CSE department homepage, the browser contacts **three different servers** to render the page:

Server	Content Provided
cs.washington.edu	The **main page** (HTML document)
youtube.com	An **embedded video** that appears as part of the page
google-analytics.com	An invisible tracking script ("nothing that the user can see but tracks visitors to the site")

How the browser integrates this content:

The browser parses the main HTML page from cs.washington.edu, discovers references to external objects (e.g., a <video> or <iframe> tag pointing to YouTube, or a script tag for Google Analytics), and then **automatically issues separate HTTP requests** to those servers. The content returned from each server is **integrated for display by the browser**-the video is rendered inline, the tracker executes silently in the background, and the result is presented as a single cohesive page to the user.

Protocol used for all fetch operations:

HTTP (HyperText Transfer Protocol). Tanenbaum explicitly states: "The request-response protocol for fetching pages is a simple text-based protocol that runs over TCP... It is called HTTP." Every object-whether from the primary server or a third-party server-is fetched using HTTP over TCP.

D. HTTP - HyperText Transfer Protocol (Section 7.3.4, before Message Headers)

Q17. HTTP as a Request-Response Protocol

HTTP is described as a "simple text-based protocol that runs over TCP." In the context of fetching a single web object, identify the **client** and **server** in the HTTP transaction, describe the basic structure of an HTTP request and an HTTP response, and explain why HTTP is called a **stateless** protocol. (Reference: Section 7.3.4, p. 683)

Answer:

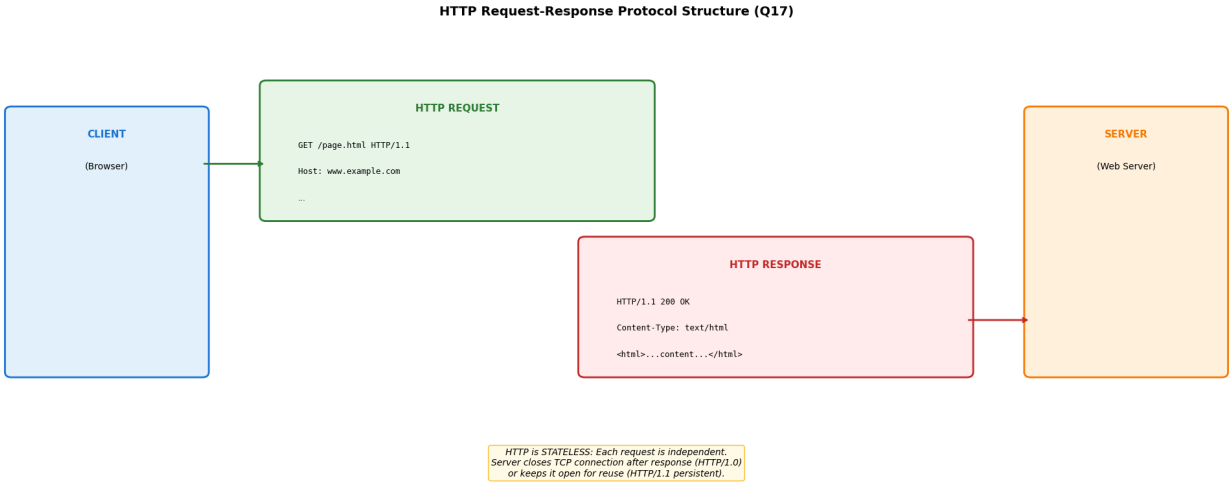

Figure 22: *HTTP Request-Response Protocol Structure*

Client and server:

- **Client:** The **browser** initiates the HTTP transaction by establishing a TCP connection to the server and sending a request message.
- **Server:** The **web server** listens for incoming TCP connections (typically on port 80), parses the request, locates or generates the requested resource, and returns a response message.

Basic structure:

HTTP is "a simple request-response protocol that normally runs over TCP." Messages consist of:

- **Request:** An ASCII command line (e.g., GET /hypertext/WWW/TheProject.html HTTP/1.1) followed by header lines, a blank line, and optionally a body.
- **Response:** A status line (e.g., HTTP/1.1 200 OK) followed by header lines, a blank line, and the requested content.

The protocol specifies "what messages clients may send to servers and what responses they get back in return. The request and response headers are given in ASCII, for ease of use by people who like to telnet to port 80 and type commands manually. The contents are given in a MIME-like format."

Why HTTP is stateless:

HTTP is **stateless** because the server treats each request as an independent transaction with no memory of previous requests. In HTTP 1.0, the server closes the TCP connection immediately after sending the response, so no session state can persist across requests. Even with persistent connections in HTTP 1.1, the server processes each incoming request independently-it does not inherently remember whether the same client requested another page two seconds ago. Any state (such as login status) must be managed by the application layer using cookies or other mechanisms outside the core HTTP protocol.

Q18. Persistent vs. Non-Persistent Connections

Early HTTP (1.0) used **non-persistent connections**, requiring a separate TCP connection for each object on a page. Explain the client-server overhead problem this creates when a page contains 10 images. How does **persistent HTTP** (1.1) solve this problem from the transport-layer perspective? What does the client need to do differently to signal the end of a persistent request sequence? (Reference: Section 7.3.4, p. 683-686)

Answer:

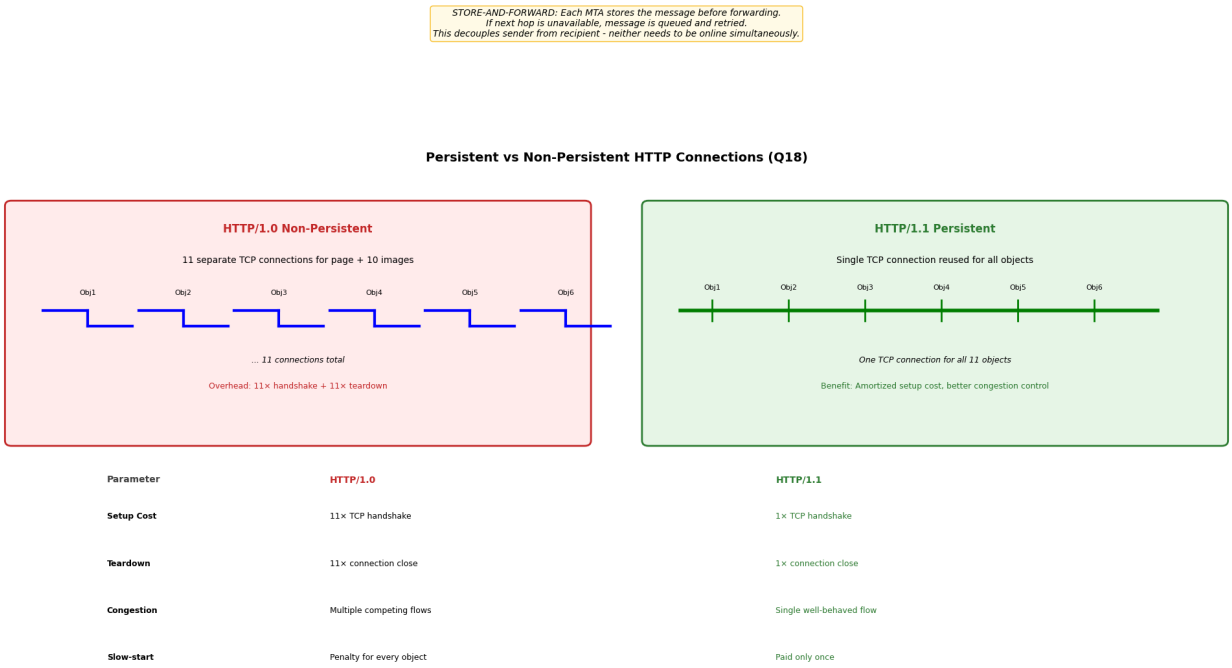

Figure 23: *Persistent vs Non-Persistent HTTP Connections*

Non-Persistent HTTP 1.0:

In early HTTP 1.0, "after the connection was established a single request was sent and a single response was sent back. Then the TCP connection was released." For a page containing the main HTML document plus 10 images, this requires **11 separate TCP connections** in sequence (or parallel), each incurring its own three-way handshake, slow-start, and teardown overhead.

Persistent HTTP 1.1:

HTTP 1.1 supports **persistent connections** (also called connection reuse). "With them, it is possible to establish a TCP connection, send a request and get a response, and then send additional requests and get additional responses" over the same TCP connection. The browser fetches the HTML page, then reuses the same connection to request embedded images and other objects.

Transport-layer benefit:

Persistent connections solve the overhead problem by **amortizing the TCP setup, startup, and release costs** across multiple requests. From the transport-layer perspective:

- TCP congestion control operates on a single connection rather than having multiple simultaneous connections "compete against each other, causing added packet loss." A single well-behaved connection is more network-friendly than many short-lived connections.
- The connection avoids the **slow-start penalty** for every object; after the first request, the connection is already in the congestion avoidance phase and can transfer subsequent objects more efficiently.

What the client does differently:

In non-persistent mode, the connection closes automatically after each response, so the client knows the transaction is finished when the TCP connection terminates. In persistent mode, the client must **keep the TCP connection open** and explicitly decide when it has finished requesting all objects for the current page. As Tanenbaum notes, "A new issue that they raise is when to close the connection." The client does not close the connection after the first response; instead, it typically pipelines additional requests or waits for user interaction. In practice, "clients and servers usually keep persistent connections open until they have been idle for a short time (e.g., 60 seconds)" or until resource limits force a closure.

Cross-Cutting Analytical Questions

Q19. Layered Client-Server Evolution

Compare the client-server interaction models of **UDP-based DNS queries**, **TCP-based SMTP message relay**, and **HTTP over TCP** in terms of: (a) transport protocol choice and justification, (b) whether the application protocol is request-response or stream-oriented, and (c) the duration of the underlying transport connection relative to the application transaction. (Synthesizes Section 6.4.1, Section 7.1, Section 7.2.4, Section 7.3.4)

Answer:

Figure 24: *Cross-Cutting Comparison: DNS vs SMTP vs HTTP*

Dimension	DNS over UDP	SMTP over TCP	HTTP over TCP

Transport protocol choice	**UDP** - because DNS queries and responses are typically small enough to fit in a single packet; UDP's minimal 8-byte header and lack of connection setup overhead make lookups fast and efficient.	**TCP** - because email messages may be arbitrarily large and must be delivered reliably without loss, duplication, or reordering. TCP provides retransmissions, flow control, and congestion control on behalf of the application.	**TCP** - because web pages, images, and embedded objects can be large and require reliable, in-order byte-stream delivery. TCP also supports persistent connections to amortize setup costs across multiple objects.
Request-response vs. stream-oriented	**Request-response.** The resolver sends a single query datagram and receives a single response datagram. Each transaction is independent and self-contained.	**Stream-oriented (over a session).** SMTP clients and servers engage in a multi-command dialogue (HELO, MAIL FROM, RCPT TO, DATA, QUIT) over a single TCP connection. Text lines flow bidirectionally in a continuous stream.	**Request-response (over a connection).** The browser sends a request message; the server returns a response message. While TCP provides a byte stream, HTTP structures the exchange as discrete request-response pairs.
Transport connection duration	**No connection.** UDP is connectionless; there is no transport-layer connection to establish or tear down. The "duration" is limited to the time the IP datagram spends in transit.	**Duration of the mail transfer session.** The TCP connection remains open for the entire SMTP dialogue-potentially multiple minutes if large messages or many recipients are involved-then closes with a QUIT command.	**Variable.** In HTTP 1.0, the TCP connection lasts for **one request-response pair** and is immediately released. In HTTP 1.1 (persistent), the same TCP connection can be reused for **multiple sequential requests** (e.g., the HTML page plus its embedded images) and is typically closed only after a short idle timeout.

Q20. Port Numbers Across the Syllabus

The following well-known port numbers appear in your syllabus scope: **53 (DNS)**, **25 (SMTP)**, and **80 (HTTP)**. Explain why these server ports are standardized and well-known, while the client-side ports (e.g., the port assigned to your browser when fetching a page) are ephemeral and dynamically allocated by the operating system. How does this design decision simplify the implementation of server software? (Synthesizes Section 6.1.1, Section 6.4.1, Section 7.1, Section 7.2.4, Section 7.3.4)

Answer:

Figure 25: *TSAPs (Ports): Well-Known Server Ports vs Ephemeral Client Ports*

Why server ports are standardized and well-known:

Server ports are fixed and published so that clients know **where to send their initial connection requests** without any prior negotiation or discovery protocol. As Tanenbaum explains, the server TSAP (e.g., TCP port 53 for DNS, 25 for SMTP, 80 for HTTP) is "permanently associated with some service and listed in a file on the server, so the server can listen on it (e.g., TCP port 25 is used for email)." If these ports were randomly assigned, a client trying to reach a web server would have no way to know which port to use.

Why client ports are ephemeral and dynamically allocated:

User processes do not have TSAP addresses that are known in advance, and they often exist for only a short time. The OS assigns an **ephemeral port** (typically from a high-numbered range, e.g., 49152-65535) to the client socket just for the duration of the transaction. The source port is needed "primarily when a reply must be sent back to the source"-the server copies this ephemeral port into the destination port of its reply so the response reaches the correct client process.

How this simplifies server software:

Because the server port is well-known, the server needs only to:

1. **Bind to one fixed port** (e.g., bind(sock, 80)) and listen there.
2. Accept incoming connections from any client, using the client's ephemeral source port to identify the return path.

The server does not need a registration service, a discovery protocol, or logic to coordinate port numbers with clients. It simply waits on its stable, published port and lets the transport layer demultiplex traffic based on the destination port. This design decision makes server implementation straightforward and allows any client anywhere on the Internet to reach the service without prior configuration.